42 COMMON CORE | PROJECT DEFENSE

cub3D: 3D Raycasting Engine in C

Building a first-person perspective from a 2D tile grid using Digital Differential Analysis (DDA)

🕹️ INSPIRATION

Wolfenstein 3D (1992)

Created by John Carmack at id Software. Ran full 3D first-person rendering smoothly on 286/386 hardware without 3D graphics acceleration cards.

📐 CORE TECHNIQUE

Raycasting with DDA

Casting 1 ray per vertical screen column, finding exact grid intersections in discrete mathematical steps, and projecting textured vertical wall slices.

🎯 EVALUATOR GUIDE

First-Principles Explanation

Assuming an evaluator with basic C knowledge (pointers, structs, loops), explaining all parsing, geometric vectors, texture mapping, and memory safety.

PART 1: FOUNDATIONAL CONCEPT

How 2D Becomes 3D: Raycasting vs True 3D

Transforming an O(N³) polygon problem into an O(Width) 2D ray computation

🧠 The Core Illusion

• The map is strictly 2D: a matrix where '1' = wall and '0' = walkable floor.

• The player is a 2D coordinate (pos_x, pos_y) with a viewing angle.

• Instead of slow 3D polygon math (z-buffers, rasterization), we cast 1 ray per vertical screen column (800 rays for 800x600).

• When a ray hits a wall at perpendicular distance D, line height = Screen Height / D.

• Closer walls appear tall; distant walls appear small. Full 3D depth perception!

⚡ Why Raycasting Was Revolutionary

• Complexity: Evaluates only visible columns O(Screen Width) rather than millions of 3D triangles.

• Constant Frame Time: Highly deterministic loop (x = 0 to 800) with minimal branching.

• No GPU Needed: Computes directly in CPU memory and writes pixels straight to RAM.

• Seamless Motion: Smooth 60+ FPS navigation achieved on simple hardware.

PART 2: PROGRAM ARCHITECTURE

System Architecture: From Main to Loop

Modular structure dividing parsing, geometry, graphics, and controls

🚀 Phase 1: Initialization & Validation

• main.c: Checks argc == 2 and initializes t_game struct.

• check_file_extension(): Validates .cub file suffix.

• read_file(): Loads all lines into memory via get_next_line.

• parse_element(): Stores textures (NO, SO, WE, EA) & RGB colors (F, C).

• validate_map_enclosed(): BFS flood-fill ensures closed walls.

• find_player(): Sets spawn coordinates (x+0.5, y+0.5) and direction.

🎮 Phase 2 & 3: Graphics & Game Loop

• init_render(): mlx_init(), creates 800x600 window, allocates screen framebuffer img & addr.

• load_textures(): Converts 4 .xpm files into memory pixel buffers.

• render_frame(): Loop hook executing every tick:
  1. handle_keys(): Player strafing, forward/back, and rotation
  2. draw_floor_and_ceiling(): Solid RGB color fills
  3. raycast_walls(): 800 rays with DDA wall projection
  4. mlx_put_image_to_window(): Pushes image buffer once per frame!

PART 3: DATA PARSING

Map Parsing & Bitwise Color Packing

Extracting configuration elements and packing RGB into 32-bit hex integers

📄 .cub Format Anatomy

• Textures: NO, SO, WE, EA paths to valid .xpm files.

• Colors: F (Floor) and C (Ceiling) formatted as R,G,B [0, 255].

• Map Matrix: Must appear last. Valid characters: '0', '1', ' ', 'N', 'S', 'E', 'W'.

• Strict Validation: Disallows missing elements, duplicate identifiers, or non-numeric tokens.

⚙️ RGB Bitwise Packing (0x00RRGGBB)

In C graphics, colors are packed into one 32-bit integer:

int color = (r << 16) | (g << 8) | b;

Example for R=220 (0xDC), G=100 (0x64), B=0 (0x00):
• r << 16 : 00000000 11011100 00000000 00000000
• g << 8 : 00000000 00000000 01100100 00000000
• b   : 00000000 00000000 00000000 00000000
• Bitwise |: 00000000 11011100 01100100 00000000
Result: 0x00DC6400 (exact 32-bit color)

PART 4: MAP VALIDATION

Map Enclosure: Iterative BFS Flood Fill

Preventing stack overflows and distinguishing legal padding from illegal holes

⚠️ The Recursive Overflow Problem

• Naive recursive flood_fill() pushes a stack frame for every tile.

• On large open maps (e.g. 100x100 = 10,000 cells), call depth exceeds the default stack limit -> SEGFAULT!

• Our Fix: Iterative Queue (BFS) allocated on the heap (t_queue).

• Enqueues player spawn, pops queue head, tests 4 cardinal neighbors (Up, Down, Left, Right). Zero recursion risk!

🛡️ Inner Spaces vs Outer Spaces

• In .cub files, outer padding spaces are legal, but holes inside walls are strictly forbidden.

• Two-Pass Verification Algorithm:
1. mark_outer_spaces(): BFS starting from borders marks true outside spaces as 'X'.
2. flood_fill_bfs(): BFS from player position. If it touches 'X' -> unenclosed wall boundary; if it touches raw ' ' -> illegal internal hole!

PART 5: CAMERA GEOMETRY

The Vector Camera Model (FOV & Direction)

Using 2D linear algebra to avoid slow trigonometric calculations for every ray

🧭 Two Vectors Define the Camera

• Direction Vector (dir_x, dir_y): Points where player is looking. Normalized length = 1.0.

• Camera Plane Vector (plane_x, plane_y): Perpendicular to direction vector. Represents the screen surface!

• Field of View (FOV):
FOV = 2 * atan(|plane| / |dir|)
Setting |dir| = 1.0 and |plane| = 0.66 yields:
FOV = 2 * atan(0.66) ≈ 66° (standard 90s FPS FOV)

📐 Ray Vector Generation

For every screen column x (from 0 to SCREEN_WIDTH - 1):

1. camera_x = 2 * x / (double)SCREEN_WIDTH - 1;
(camera_x ranges from -1.0 on left to +1.0 on right)

2. ray_dir_x = dir_x + plane_x * camera_x;
ray_dir_y = dir_y + plane_y * camera_x;

No sin() or cos() needed per ray! Linear combination creates the entire ray fan instantly.

PART 6: RAYCASTING ENGINE

Digital Differential Analysis (DDA) Algorithm

Exact grid line intersection testing without stepping or missing thin corners

❌ Why Fixed Steps (dx = 0.05) Fail

• Moving in small increments is slow (hundreds of loops per ray).

• If step size is too large, rays jump straight through wall corners (tunneling bug).

• If step size is too small, performance drops severely in large rooms.

• Prone to floating-point rounding errors.

✅ The DDA Grid-Hopping Solution

• Map cells are 1.0 x 1.0 integer units.

• Delta Distance: Distance ray must travel to cross 1 full grid line:
  delta_dist_x = |1 / ray_dir_x|
  delta_dist_y = |1 / ray_dir_y|

• Step: At each iteration, compare side_dist_x vs side_dist_y. Advance by whichever is smaller.

• Guarantees 100% precision: every wall boundary is hit with minimum operations!

PART 7: RAYCASTING MATH

Eliminating the Fisheye Effect

Why Euclidean distance distorts flat walls into spheres and how perpendicular distance fixes it

🐟 The Fisheye Problem

• If distance is computed using the Pythagorean theorem: dist = sqrt((x_w - x_p)² + (y_w - y_p)²)

• Rays hitting the edge of the screen travel farther than rays hitting the center, even on a perfectly flat wall!

• Walls farther away are drawn shorter. Result: A flat straight wall appears curved like a fishbowl!

📐 Perpendicular Distance Projection

• We project the Euclidean distance onto the viewing direction vector (cos(theta) correction):

```
if (side == 0)
    perp_dist = side_dist_x - delta_dist_x;
else
    perp_dist = side_dist_y - delta_dist_y;
```

• Orthogonal distance eliminates distortion: straight walls remain completely straight on screen!

PART 8: 2D PROJECTION

Wall Projection: Calculating Column Slices

Transforming 3D perpendicular distance into 2D vertical screen pixels

📏 Calculating Vertical Slice Bounds

For each screen column x (from 0 to 799):

1. line_height = (int)(SCREEN_HEIGHT / perp_dist);

2. draw_start = -line_height / 2 + SCREEN_HEIGHT / 2;
   if (draw_start < 0) draw_start = 0;

3. draw_end = line_height / 2 + SCREEN_HEIGHT / 2;
   if (draw_end >= SCREEN_HEIGHT) draw_end = SCREEN_HEIGHT - 1;

Clamping prevents drawing outside screen memory buffers.

🖼️ The 3 Vertical Screen Zones

Every column x is divided into three distinct vertical zones:

1. Ceiling Zone (y = 0 to draw_start - 1):
   Filled with solid ceiling_colour

2. Wall Zone (y = draw_start to draw_end):
   Sampled from corresponding wall texture

3. Floor Zone (y = draw_end + 1 to SCREEN_HEIGHT - 1):
   Filled with solid floor_colour

PART 9: TEXTURE MAPPING

Wall Texturing: Mapping XPM Pixels

Determining face orientation and mapping texture coordinates (tex_x, tex_y)

🧭 1. Face Orientation Detection

From DDA hit data (side = 0 for EW, side = 1 for NS):

```
int get_tex_idx(t_game *game) {
    if (game->ray.side == 0) {
        if (game->ray.ray_dir_x > 0)
            return (3); // EA (East)
        return (2);     // WE (West)
    }
    if (game->ray.ray_dir_y > 0)
        return (1);     // SO (South)
    return (0);         // NO (North)
}
```

🎨 2. Sampling (tex_x, tex_y)

- Hit coordinate on wall (wall_x):

```
wall_x = pos + perp_dist * ray_dir;
wall_x -= floor(wall_x); // fractional part [0, 1)
tex_x = (int)(wall_x * tex->width);
```

- Vertical scaling:

```
step = 1.0 * tex->height / line_height;
tex_pos = (draw_start - SCREEN_HEIGHT / 2 +
line_height / 2) * step;
```

- Address pixel color directly from memory address!

PART 10: GRAPHICS PIPELINE

MiniLibX: Software Double Buffering

Writing directly to RAM pixel buffers to eliminate screen tearing and flickering

🐌 The Naive mlx_pixel_put() Trap

• Calling mlx_pixel_put(mlx, win, x, y, color) for every pixel:

• Sends an X11 / Cocoa IPC packet per pixel!

• 800 x 600 = 480,000 system calls per frame.

• Result: Unplayable ~2 FPS and severe visual flickering.

⚡ Direct Framebuffer Writing

• Allocate one contiguous screen image buffer:
img = mlx_new_image(mlx, 800, 600);
addr = mlx_get_data_addr(img, &bpp, &line_len, &endian);

• my_mlx_pixel_put(): Writes directly to memory:
dst = addr + (y * line_len + x * (bpp / 8));
*(unsigned int *)dst = color;

• mlx_put_image_to_window() called ONCE per frame: Locked 60+ FPS!

PART 11: CONTROLS & PHYSICS

Movement, Collision & Wall Sliding

Smooth navigation physics and 2D rotation matrix transformations

🎮 Wall Sliding Physics

• When moving forward, backward, or strafing:

• Independent Axis Testing: Check X coordinate and Y coordinate separately.

• Collision buffer: buf = 0.1 prevents clipping.

• If moving diagonally into a wall, the blocked axis stops, but the free axis continues moving.

• Result: The player smoothly slides along walls instead of sticking abruptly!

🔄 2D Vector Rotation Matrix

Turning left or right with arrow keys applies the rotation matrix to both direction and camera plane vectors:

```
[ cos(θ)  -sin(θ) ] * [ x ]
[ sin(θ)   cos(θ) ]   [ y ]
```

```
old_dir_x = dir_x;
dir_x = dir_x * cos(rot) - dir_y * sin(rot);
dir_y = old_dir_x * sin(rot) + dir_y * cos(rot);
```

(Same formula applied to plane_x and plane_y)

PART 12: BONUS FEATURES

Bonus: Real-Time Minimap Overlay

HUD radar display with strict 42 bonus isolation in cub3D_bonus

🗺️ Minimap Specifications

• Position: Top-left HUD overlay.

• Circular Field-of-View: Bounded radius of 6 tiles around player (MM_RAD = 6).

• Scale: 10 pixels per cell (MM_SCALE = 10).

• Legend: Dark Gray = Wall ('1'), Light Gray = Floor ('0'), Black = Outside, Red 2x2 = Player position.

• Player stays centered while radar tracks movement.

📦 42 Bonus Isolation Compliance

• Strict separation: minimap_bonus.c is isolated.

• Mandatory (cub3D): Compiled without minimap code.

• Bonus (cub3D_bonus): Activated via -DBONUS=1.

• Dedicated object directories (obj/ vs obj_bonus/) prevent object file contamination.

• Full compliance with 42 Common Core evaluation rules.

PART 13: MEMORY SAFETY

Memory Lifecycle & Zero-Leak Architecture

Guaranteed clean termination under Valgrind across all exit triggers

🧹 Centralized Cleanup: close_game()

• Single unified exit handler:

1. Free 2D map array (free_2d_array)

2. Free 4 texture paths and destroy XPM images (mlx_destroy_image)

3. Destroy screen buffer image (mlx_destroy_image)

4. Destroy window (mlx_destroy_window) and display (mlx_destroy_display)

5. Free mlx pointer and exit(0)

✅ Valgrind Audit Report

Tested extensively with Valgrind Memcheck:

```
== HEAP SUMMARY:
==     in use at exit: 0 bytes in 0 blocks
==   total heap usage: 171 allocs, 171 frees
==
== All heap blocks were freed -- no leaks possible
== ERROR SUMMARY: 0 errors from 0 contexts
```

• ESC key and window [X] unified into close_game().

PART 14: PEER DEFENSE GUIDE

Top Peer Evaluation Questions & Answers

Concise, precise explanations for the most common questions evaluators ask

Q1: What is DDA and why did you choose it?

DDA (Digital Differential Analysis) calculates the exact mathematical distance to the next grid line (X or Y), jumping directly from boundary to boundary. Unlike fixed floating-point steps (0.01), it guarantees 100% precision and never tunnels through wall corners.

Q2: Why use perpendicular distance instead of Euclidean distance?

Euclidean distance measures the hypotenuse from the player to the hit point. Since diagonal rays travel farther to reach the same flat wall than rays facing straight ahead, Euclidean distance bends flat walls into spherical fishbowls. Perpendicular distance keeps walls flat.

Q3: How does your flood fill check maps without crashing on big maps?

Recursive flood fill creates a new stack frame per cell, overflowing the stack on large maps. We implemented an iterative Breadth-First Search (BFS) using a heap-allocated queue. It marks external border spaces first, then verifies the player never touches open boundaries.

Q4: How do you prevent memory leaks when pressing ESC or on error?

Both the window 'X' event and KEY_ESC are unified into close_game(), destroying all MLX images, windows, displays, texture paths, and 2D arrays. In parsing routines, all split buffers are freed before calling error_exit(), verified by Valgrind (0 leaks, 0 errors).